

Skema Teknis & Panduan Implementasi Prototype CCTV Privat dengan ESP-32-CAM

Latar Belakang

Penggunaan CCTV (*Closed Circuit Television*) berbasis IP atau *Smart Camera* di area perumahan telah menjadi standar baru dalam sistem keamanan mandiri. Namun, sebagian besar perangkat CCTV komersial yang beredar di pasaran saat ini sangat bergantung pada ekosistem *cloud* pihak ketiga atau terikat pada batasan jaringan lokal (*Local Area Network*). Ketergantungan pada penyedia layanan *cloud* komersial sering kali menimbulkan kekhawatiran terkait privasi data, potensi kebocoran rekaman, serta biaya langganan bulanan yang relatif mahal. Di sisi lain, jika sistem CCTV dikonfigurasi murni lokal tanpa jembatan internet, pengguna akan kehilangan kemampuan esensial untuk memantau kondisi rumah dari jarak jauh saat sedang bepergian.

Project ini hadir sebagai solusi alternatif dengan mereplikasi infrastruktur CCTV pintar yang mandiri, aman, dan sepenuhnya dikendalikan oleh pengguna (*self-hosted*). Dengan memanfaatkan mikrokontroler ekonomis ESP32-CAM sebagai penangkap gambar dan VPS (Virtual Private Server) Linux berbasis internet sebagai server perantara, sistem ini mampu mematahkan batasan geografis tanpa mengorbankan kedaulatan data. Aliran video (*streaming data flow*) akan dikirimkan langsung dari rumah menuju VPS pribadi milik pengguna, sehingga pemantauan *live feed* dapat diakses dengan aman dari mana saja melalui browser web. Melalui pendekatan integrasi backend Golang, kontainerisasi Docker, dan skrip otomasi Bash (**install.sh**), seluruh arsitektur server kompleks ini dapat direplikasi dan diinstalasi dengan sangat mudah hanya dalam satu perintah perintah di lingkungan Linux.

Sebagai **disclaimer** untuk menghindari **kesalahpahaman**, kata "**privat**" dalam project ini bukan berarti sistem CCTV ini hanya bekerja secara **offline** di dalam jaringan Wi-Fi lokal rumah saja. Perangkat ini tetap terkoneksi penuh ke jaringan internet global, namun dikategorikan sebagai **privat** karena seluruh jalur data dan infrastruktur servernya berada di bawah kepemilikan dan kendali mutlak pengguna melalui VPS pribadi, bukan menumpang pada *cloud* publik milik vendor pihak ketiga. Oleh karena itu, **prototype** ini akan jauh lebih efisien dan optimal jika diimplementasikan pada lingkungan atau oleh pengguna yang sudah familier dengan konsep dasar dunia IT, khususnya dalam hal pengelolaan dan konfigurasi sebuah VPS Linux.

1. Daftar Alat & Komponen (Bill of Materials)

- **ESP32-CAM Modul + Kamera OV2640:** Unit utama yang berfungsi sebagai pemroses data, modul Wi-Fi, dan sensor penangkap gambar.



- **ESP32-CAM-MB Shield:** Board bawah yang dilengkapi IC CH340/FT232. Berfungsi untuk *flashing code* via USB sekaligus sebagai regulator daya operasional.



- **Tampilan Setelah Dirakit:**



- **Kabel Data USB (Micro USB / Type-C):** Menghubungkan *burning stand* ke komputer (saat *coding*) atau ke adaptor (saat diletakkan sebagai CCTV).



- **Adaptor Charger HP 5V (Minimal 1A - 2A):** Sumber daya utama agar CCTV dapat menyala secara mandiri (*stand-alone*) tanpa komputer.



2. Skema Koneksi Perangkat

Karena akan menggunakan modul ESP32-CAM-MB, proses perakitan fisiknya sangat praktis karena semua modul dirancang untuk saling terhubung secara langsung tanpa memerlukan papan sirkuit tambahan (*breadboard* atau PCB) maupun kabel jumper.

A. Pemasangan Kamera ke Board Utama

- Ambil modul kamera OV2640 (yang memiliki kabel pita fleksibel berwarna kuning tembaga).
- Buka kunci klip hitam pada soket pita di board ESP32-CAM dengan hati-hati.
- Masukkan ujung kabel pita kamera ke dalam soket tersebut dengan posisi pin/jalur tembaga menghadap ke arah board.
- Tekan kembali klip hitam untuk mengunci kabel pita kamera agar posisinya kuat dan tidak bergeser.

B. Penggabungan Board ESP32-CAM dan Shield MB

- Posisikan board ESP32-CAM di atas board ESP32-CAM-MB (*burning stand* bagian bawah).

- Sejajarkan pin-pin *male* (kaki jarum) pada ESP32-CAM dengan lubang pin *female* (soket) yang ada di board MB. Pastikan posisinya pas dan tidak terbalik (biasanya ditandai dengan posisi modul antena Wi-Fi dan *port* USB yang berada di sisi yang searah).
- Tekan board ESP32-CAM ke bawah secara perlahan hingga seluruh kaki pin tertancap sempurna ke dalam soket board MB.

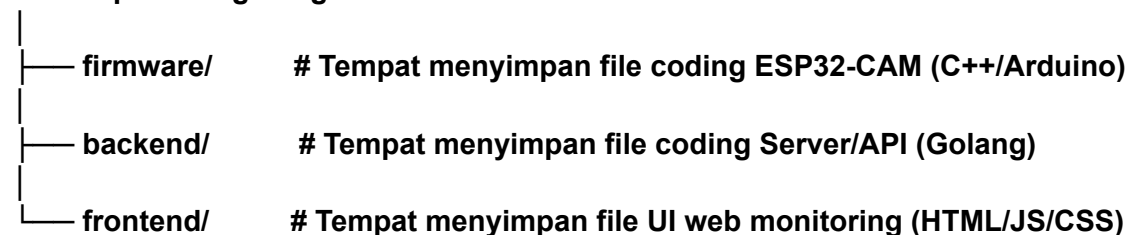
C. Jalur Data dan Koneksi Daya (Dua Fase Operasional)

Setelah kedua board menyatu, kemudian hanya perlu memanfaatkan satu *port* USB (Micro USB atau Type-C) yang ada pada board MB terbawah untuk dua kondisi berikut:

- **Fase Pengembangan (Coding & Flashing):** Hubungkan kabel data USB dari *port* pada board MB langsung ke Port USB Laptop/Komputer. Pada fase ini, komputer akan menyuplai daya sekaligus menjadi jalur untuk memasukkan kode program (*firmware*) dari Arduino IDE ke dalam chip ESP32.
- **Fase Operasional (Deploy Stand-alone):** Setelah kode selesai dimasukkan dan CCTV siap dipasang, cabut kabel dari komputer. Hubungkan kabel USB tersebut dari board MB ke Adaptor Charger HP 5V (minimal 1A–2A), lalu colokkan adaptor ke stopkontak listrik. CCTV sekarang dapat menyala secara mandiri dan langsung terhubung ke Wi-Fi tanpa perlu komputer lagi.

3. Struktur Folder Project (Monorepo)

cctv-esp32cam-golang/



4. Alur Logika & Arsitektur Data (MJPEG Streaming)

Project ini menggunakan metode Motion JPEG (MJPEG). Secara konsep, alur datanya berjalan linear dari perangkat keras menuju server, lalu didistribusikan ke pengguna.

A. Tahap Pertama: Inisialisasi dan Koneksi Perangkat

Proses dimulai saat ESP32-CAM dinyalakan. Firmware di dalam ESP32-CAM akan mengaktifkan driver kamera OV2640 dan mengatur resolusi gambar (misalnya VGA). Setelah kamera siap, modul Wi-Fi pada ESP32-CAM akan mencari sinyal dan menghubungkan diri ke *access point* lokal. Ketika koneksi internet lokal berhasil tersambung, ESP32-CAM akan

melakukan jabat tangan (*handshake*) awal dengan server backend Golang untuk memastikan server siap menerima kiriman data.

B. Tahap Kedua: Pengiriman Data Gambar secara Kontinu (Looping)

Setelah terhubung, ESP32-CAM masuk ke dalam siklus *looping* tanpa henti. Kamera akan menangkap gambar baru dan menyimpannya di dalam memori internal (*frame buffer*) sebagai data biner berformat JPEG. Perangkat kemudian langsung mengirimkan data biner tersebut ke endpoint upload server Golang menggunakan metode HTTP POST. Setelah data sukses terkirim, firmware akan mengosongkan kembali *frame buffer* kamera agar siap menampung gambar berikutnya pada siklus berikutnya. Proses ini berulang sangat cepat demi menghasilkan transisi video yang halus.

C. Tahap Ketiga: Pemrosesan dan Penyaluran Data oleh Server Golang

Di sisi server, backend Golang bersiap dengan HTTP Server yang berjalan terus-menerus. Golang menyediakan dua fungsi utama. Fungsi pertama bertugas menerima setiap kiriman data gambar mentah dari ESP32-CAM. Karena Golang sangat efisien dalam menangani banyak proses sekaligus (*concurrency*), data yang masuk akan langsung dialirkan menggunakan mekanisme *channel* ke fungsi kedua. Fungsi kedua ini bertugas menyiarkan (*broadcast*) kumpulan gambar tersebut ke halaman web monitoring menggunakan *header* HTTP khusus. Koneksi ini dijaga agar tetap terbuka tanpa pernah terputus.

D. Tahap Keempat: Tampilan Dashboard pada Web Monitoring

Di ujung alur, aplikasi web frontend bertindak sebagai monitor. Browser pengguna cukup mengakses alamat server Golang yang menyiarkan video tersebut. Karena menggunakan metode MJPEG, browser secara otomatis mengenali aliran data tersebut sebagai kumpulan gambar yang bergerak. Halaman web tidak memerlukan kode JavaScript yang kompleks atau berat; cukup menggunakan tag gambar standar untuk langsung merender dan menampilkan video CCTV secara *real-time* kepada pengguna.

5. Setup Docker & Skrip Otomasi Deployment (Bash Script)

Poin ini dirancang agar pengguna tidak perlu menginstall Golang secara manual di sistem mereka. Semua *dependency* backend dan frontend akan dibungkus di dalam kontainer Docker, sementara firmware ESP32-CAM tetap di-upload manual dari lokal melalui Arduino IDE.

A. Mekanisme Dockerization

Project ini akan memanfaatkan fitur **Docker Multi-stage Build** untuk backend Golang. Pada tahap pertama, Docker akan meminjam *image* resmi Golang untuk mengompilasi kode menjadi sebuah file biner yang sangat ringan. Setelah kompilasi selesai, file biner tersebut dipindahkan ke *image* Linux yang bersih dan minimalis (seperti Alpine Linux), lalu digabungkan dengan file statis dari folder frontend. Hasil akhirnya adalah sebuah kontainer Docker yang sangat kecil, efisien, dan hemat RAM (sangat cocok untuk laptop Celeron).

B. Konfigurasi Docker Compose

Untuk mempermudah manajemen kontainer, project ini menyediakan file konfigurasi Docker Compose. File ini bertugas untuk mendefinisikan bagaimana kontainer backend berjalan, mengatur pemetaan *port* jaringan (misalnya memetakan port 8080 di dalam kontainer ke port 8080 laptop agar bisa diakses oleh ESP32-CAM dan browser), serta memastikan kontainer otomatis menyala kembali jika laptop melakukan *restart*.

C. Cara Kerja Skrip Otomasi Bash (install.sh)

Ketika pengguna menjalankan skrip Bash ini di terminal Ubuntu, skrip akan melakukan serangkaian validasi dan eksekusi otomatis.

1. Pertama, skrip akan memeriksa apakah Docker dan Docker Compose sudah terinstal di sistem. Jika belum, skrip akan memberikan panduan atau otomatis memasangnya menggunakan *package manager* bawaan Ubuntu (*apt*).
2. Kedua, skrip akan menyalin file konfigurasi lingkungan (seperti pengaturan port atau password default jika ada).
3. Ketiga, skrip akan langsung menjalankan perintah pembuatan dan pengaktifan kontainer di latar belakang. Setelah proses selesai, skrip akan menampilkan pesan sukses di terminal beserta informasi URL IP Address lokal yang harus dimasukkan ke dalam kode firmware ESP32-CAM.

Untuk mendukung Poin 5 ini, struktur folder *plain text* pada project diatas akan sedikit berubah dengan penambahan beberapa file konfigurasi di folder utama:

cctv-esp32cam-golang/

— firmware/	# Tempat menyimpan file coding ESP32-CAM (C++/Arduino)
— backend/	# Tempat menyimpan file coding Server/API (Golang)
— frontend/	# Tempat menyimpan file UI web monitoring (HTML/JS/CSS)
— Dockerfile	# Instruksi untuk membuat image Docker backend & frontend
— docker-compose.yml	# Konfigurasi untuk menjalankan kontainer CCTV
— install.sh	# Skrip Bash untuk instalasi satu perintah di Ubuntu

Kesimpulan

Project rancang bangun CCTV privat berbasis ESP32-CAM dan Golang ini berhasil membuktikan bahwa sistem monitoring keamanan jarak jauh yang andal dapat dibangun secara mandiri tanpa ketergantungan pada vendor *cloud* pihak ketiga. Penggunaan metode *MJPEG Streaming* yang efisien, dipadukan dengan performa tinggi dari *concurrency* backend Golang, memastikan konsumsi sumber daya (RAM dan CPU) tetap minimal, sehingga sistem ini sangat ideal untuk dijalankan pada VPS Linux berspesifikasi rendah sekalipun.

Melalui penerapan arsitektur *self-hosted* yang dijembatani oleh VPS pribadi, pengguna mendapatkan fleksibilitas penuh untuk memantau area rumah dari jarak jauh secara *real-time*, sekaligus memegang kendali mutlak atas keamanan dan kerahasiaan data video mereka. Ditambah dengan implementasi Docker dan skrip otomatisasi **install.sh**, kompleksitas proses instalasi infrastruktur server dapat dipangkas secara signifikan. Proses deployment satu langkah ini memberikan kemudahan bagi pengguna berskala rumahan untuk mengadopsi teknologi keamanan modern yang portabel, murah, namun memiliki standar privasi tingkat tinggi.